

Circuito compatto per il passo di Trotter

Teoria, verifica, e perché non si usa (per ora)

Note di lavoro — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Nota di riferimento su un'analisi collaterale fatta durante il lavoro sul dimero: il Prof. Chiesa ha chiesto se il circuito del passo di Trotter potesse essere reso più compatto (esempio: con 2 CNOT invece degli 8 gate a due qubit attuali). Questo file raccoglie la teoria usata, la verifica numerica fatta (con due metodi indipendenti), il risultato, e — soprattutto — perché quel risultato **non va usato ingenuamente** quando si passerà a discutere di rumore (sistema aperto) e di ricerca del ground state. Pensato per essere caricato come riferimento all'inizio di quella discussione.

1 Il problema

Il circuito attuale per un singolo passo di Trotter usa **8 gate a due qubit**: $1 \times R_{XX} + 1 \times R_{YY} + 6 \times \text{CNOT}$ (i 6 CNOT vengono dai tre blocchi R_{ZZ} scritti esplicitamente come $\text{CNOT}-R_z-\text{CNOT}$: uno per lo scambio, due per il termine DM). Profondità 15.

La domanda del relatore: si può fare con meno gate, ad esempio con 2 CNOT?

2 La teoria: quanti CNOT servono per un'unitaria a 2 qubit qualsiasi

Il risultato generale (letteratura)

Ogni operatore unitario a 2 qubit $U \in SU(4)$ si può scrivere, a meno di trasformazioni locali a un qubit (che non costano CNOT), nella forma canonica di Cartan:

$$U_{\text{local}} = \exp \left[-\frac{i}{2} (c_1 X \otimes X + c_2 Y \otimes Y + c_3 Z \otimes Z) \right], \quad 0 \leq c_3 \leq c_2 \leq c_1 \leq \frac{\pi}{2}$$

I tre numeri (c_1, c_2, c_3) — le **coordinate di Weyl** — classificano completamente la "non-località" del gate, indipendentemente da come è vestito con rotazioni a un qubit. Il numero minimo di CNOT necessari a realizzare U dipende **solo** da queste coordinate:

Condizione su (c_1, c_2, c_3)	CNOT minimi
$c_1 = c_2 = c_3 = 0$	0 (gate locale, nessun entanglement)
$(c_1, c_2, c_3) = (\pi/4, 0, 0)$	1 (classe di equivalenza locale del CNOT)
$c_3 = 0$ (ma non nel caso sopra)	2
$c_3 \neq 0$	3 (il caso generico)

Bibliografia — verificata con fonti indipendenti (non solo plausibile)

Ogni citazione qui sotto è stata controllata contro fonti terze (pagine ufficiali degli editori, bibliografie di altri lavori) prima di essere inclusa:

- **F. Vatan, C. Williams**, *Optimal Quantum Circuits for General Two-Qubit Gates*, Phys. Rev. A **69**, 032315 (2004). Dimostra che 3 CNOT bastano sempre e sono necessari nel caso generico, con costruzione esplicita indipendente. **Riferimento primario consigliato per questo teorema**: nessuna correzione nota in letteratura.

- **G. Vidal, C. M. Dawson**, *A Universal Quantum Circuit for Two-Qubit Transformations With Three CNOT Gates*, Phys. Rev. A **69**, 010301 (2004). Stesso risultato, costruzione diversa.

*Attenzione: esiste un Comment pubblicato — M. W. Coffey, R. Deiotte, T. Semi, Phys. Rev. A **77**, 066301 (2008) — che segnala errori nella costruzione esplicita presentata in questo paper. Non è stato possibile verificare il dettaglio dell'errore (testo a pagamento), ma il teorema principale (3 CNOT necessari e sufficienti nel caso generico) resta lo standard accettato: citato senza riserve in decine di lavori successivi fino al 2024–2025, ed è comunque corroborato indipendentemente dal punto precedente (Vatan–Williams) e dalla verifica numerica fatta qui sotto con il codice di Qiskit.*

- **Y. Makhlin**, *Nonlocal Properties of Two-Qubit Gates and Mixed States and Optimization of Quantum Computations*, Quantum Inf. Process. **1**, 243 (2002) — introduce gli invarianti locali (equivalenti alle coordinate di Weyl).
- **J. Zhang, J. Vala, S. Sastry, K. B. Whaley**, *Geometric theory of nonlocal two-qubit operations*, Phys. Rev. A **67**, 042313 (2003) — lega gli invarianti di Makhlin alla decomposizione di Cartan e alla "camera di Weyl".

Perché conta c_3 e non c_1, c_2

Intuitivamente: c_1 e c_2 si possono sempre "assorbire" in parte con trasformazioni locali astute; è la terza coordinata, quella più piccola nella convenzione ordinata $c_1 \geq c_2 \geq c_3$, a essere l'ostacolo residuo che nessuna rotazione locale può eliminare. Un CNOT (o CZ, equivalente a meno di locali) ha $(c_1, c_2, c_3) = (\pi/4, 0, 0)$: $c_3 = 0$ esattamente, il caso più semplice possibile per un gate davvero entanglante. Un'operazione con $c_3 \neq 0$ non può quindi mai essere raggiunta con un solo CNOT, né con due (che restano confinati a $c_3 = 0$), e servono 3.

3 Verifica del metodo su casi noti

Prima di applicare il criterio al passo di Trotter, è stato validato su quattro casi di cui il risultato è già certo, usando `qiskit.synthesis.TwoQubitWeylDecomposition`:

Gate	Coord. di Weyl (a, b, c)	CNOT minimi (atteso)	Verificato
identità	(0, 0, 0)	0	✓
CNOT	($\pi/4, 0, 0$)	1	✓
iSWAP	($\pi/4, \pi/4, 0$)	2	✓
SWAP	($\pi/4, \pi/4, \pi/4$)	3	✓

4 Applicazione al passo di Trotter del dimero

Con $b/J = -0.18$, $D/J = 1$ (il punto di lavoro adottato) e τ variabile:

$$U_{\text{step}}(\tau) = e^{-iH_2\tau} e^{-iH_1\tau}$$

τ	Weyl (a, b, c)	CNOT minimi
0.10	(0.035, 0.035, -0.025)	3
0.50	(0.173, 0.173, -0.125)	3
1.00	(0.319, 0.319, -0.250)	3
1.50	(0.391, 0.391, -0.375)	3

La terza coordinata $c \neq 0$ per **ogni** $\tau \neq 0$ testato (e per costruzione, per ogni $\tau \neq 0$ in generale, dato che $c \propto -\tau/4$ in questo caso — proporzionale al termine DM). Verificato anche nel caso $D = 0$ (senza termine DM): $c \neq 0$ comunque, quindi nemmeno l'assenza del DM riporta al caso a 2 CNOT — lo scambio isotropo da solo già impone $c \neq 0$.

Seconda verifica, con un metodo Qiskit indipendente dal primo

Oltre a `TwoQubitWeylDecomposition`, il risultato è stato ricontrollato con `TwoQubitBasisDecomposer` (un algoritmo di sintesi diverso, non solo una lettura delle coordinate), forzando esplicitamente un numero fissato di CNOT e misurando la fedeltà effettivamente raggiunta rispetto a $U_{\text{step}}(\tau = 1)$:

CNOT forzati	Fedeltà raggiunta
0	0.764
1	0.676
2	0.939
3	1.000

Fedeltà esatta **solo** a partire da 3 CNOT — mai con 2, per quanta ottimizzazione numerica si tenti. Il metodo `num_basis_gates()` di Qiskit, che implementa autonomamente questa stessa ricerca, restituisce **3** in modo indipendente.

Conclusione sulla domanda originale

2 CNOT sono matematicamente impossibili per questo passo, confermato da due algoritmi diversi. Il minimo raggiungibile è 3.

5 Dal minimo teorico (3) al circuito realizzato: due strade diverse

Comprimere il singolo passo

Il circuito attuale usa 8 gate a due qubit per passo; il minimo è 3. Il circuito compatto è stato ottenuto in tre passaggi distinti, ed è importante tenerli separati per capire cosa produce davvero:

1. **Calcolo classico preliminare:** la matrice 4×4 esatta del passo, $U_{\text{step}} = e^{-iH_2\tau}e^{-iH_1\tau}$, calcolata con `scipy.linalg.expm` — non un circuito, pura algebra lineare classica.
2. **La matrice passata come blocco numerico opaco:** `qc.unitary(U_step, [0,1])` — a questo punto Qiskit ha in mano solo 16 numeri complessi, non sa che provengono da un'Hamiltoniana con un termine di scambio e un termine DM.
3. **Sintesi via transpiler:** `transpile(qc, basis_gates=['cx','rz','sx','x'], optimization_level=3)`, che internamente usa la decomposizione di Cartan/KAK descritta in sez. 2 per ricostruire *qualunque* matrice 4×4 con il minor numero di CNOT possibile.

Il punto cruciale è nel passaggio 2-3: l'ottimizzatore non sa cosa sia H_1 , H_2 , lo scambio isotropo o il termine DM — vede solo "questa matrice 4×4 , riproducila con meno gate possibile". È pura ottimizzazione numerica sulla matrice finale, indifferente a come quella matrice si è generata. Per questo il circuito compatto che ne esce ha angoli R_z senza alcun significato fisico leggibile (non si legge più "questo pezzo è lo scambio, questo è il DM") — non è un dettaglio implementativo, è la conseguenza diretta del fatto che l'informazione sulla struttura fisica è stata scartata già al passaggio 2, quando l'Hamiltoniana è stata ridotta a una matrice di numeri senza etichette. Il file `circuito_compatto.png` mostra il risultato di questi tre passaggi, con fedeltà 1.0 rispetto al passo esatto.

Comprimere l'intero circuito a N passi — il punto concettualmente più importante

Qui emerge un fatto che a prima vista sembra sorprendente: comprimendo l'intero prodotto $U_{\text{Trotter}}(t, N) = [e^{-iH_2\tau}e^{-iH_1\tau}]^N$ (non passo per passo, ma il prodotto finale), il risultato resta

sempre un'unica unitaria a 2 qubit — e quindi si comprime anch'esso a **3 CNOT**, indipendentemente da N :

N	gate 2q (esplicito)	gate 2q (compresso)	risparmio
1	8	3	62%
2	16	3	81%
5	40	3	92%
20	160	3	98%
100	800	3	99.6%

Verificato: la fedeltà del circuito compresso rispetto a $U_{\text{Trotter}}(t, N)$ (non rispetto all'evoluzione esatta) resta 1.0 per ogni N — cioè il circuito compresso realizza **esattamente** la stessa unitaria approssimata di Trotter, solo con meno gate. L'errore di Trotter (rispetto all'evoluzione vera) è identico prima e dopo la compressione, a qualunque N : comprimere **non cambia l'accuratezza**, cambia solo quanti gate fisici servono per realizzarla.

6 Perché questa scorciatoia non regge per il lavoro con il rumore (e va dichiarata come tale)

Questo è il punto da avere chiaro **prima** di passare alla Parte 2.

Il compromesso N -vs-rumore sparisce

Tutta l'idea della Parte 2 è studiare come l'errore *algoritmico* di Trotter (che scala come $\sim 1/N$, e quindi vuole N grande) e l'errore *fisico* dei gate sotto un modello di rumore (che scala con il numero di gate, e quindi vuole N piccolo) si bilancino fra loro, per trovare un compromesso ottimale. Se si usa il circuito compresso, il costo in gate resta fisso a **3 qualunque sia N** : il compromesso a cui la Parte 2 è interessata smette di esistere nel grafico. Si misurerebbe il rumore su un'unitaria diversa dall'oggetto di studio.

La compressione richiede un calcolo classico pregresso, quindi non è "vero" Trotter

Lo stesso meccanismo descritto per il singolo passo (calcolo classico della matrice, poi sintesi) si applica identico all'intero prodotto: per ottenere il circuito a 3 CNOT bisogna **prima** calcolare classicamente $U_{\text{Trotter}}(t, N)$ come matrice 4×4 (con **scipy**), e solo *dopo* comprimerla in gate. Su un sistema più grande di 2 qubit — cioè il caso per cui Trotter è stato inventato — questo calcolo classico preliminare è esattamente quello che si vuole evitare (matrice $2^n \times 2^n$, intrattabile per n grande). La compressione funziona qui solo perché il dimero è un giocattolo a 2 qubit comodo da diagonalizzare a mano: è una scorciatoia specifica di questo caso di test, non un metodo generalizzabile a sistemi più grandi.

Dove la forma compatta avrebbe senso, con l'avvertenza giusta

Se in futuro si vuole discutere "qual è il rumore minimo possibile per realizzare *questa specifica* evoluzione a un tempo t fissato" (una domanda a sé, legittima ma diversa), il circuito compresso è lo strumento giusto. Ma va tenuto esplicitamente **separato** dal test "vero" di Trotter con rumore, che deve usare il circuito esplicito (o comunque uno il cui conteggio di gate scali con N), altrimenti si confondono due esperimenti concettualmente diversi.

7 Riepilogo per la discussione futura

Quando si affronterà il rumore su un sistema quantistico aperto (generico su X, Y, Z) e la ricerca del ground state, tenere presente:

1. **Il conteggio di gate del circuito esplicito scala linearmente con N** (8 gate a due qubit per passo, $8N$ in totale) — è quello giusto da usare per studiare il compromesso Trotter/rumore, perché riflette il costo fisico reale che si avrebbe anche su un sistema più grande.
2. **Il circuito compresso a 3 CNOT è una scorciatoia valida solo per la verifica puntuale su questo sistema-giocattolo a 2 qubit** — utile per sapere "qual è il limite fisico minimo", ma da non usare come sostituto del circuito di Trotter quando si introduce il rumore, pena perdere il fenomeno che si vuole studiare.
3. Se il lavoro sul ground state passerà per metodi variazionali (VQE) o evoluzione in tempo immaginario, la stessa distinzione tornerà utile: un circuito "compresso/ottimizzato numericamente" che riproduce un risultato target non è la stessa cosa di un circuito costruito con una struttura fisica interpretabile (ansatz), anche quando le due unitarie coincidono esattamente — la prima non generalizza, non è ispezionabile termine a termine, e non dice nulla sul comportamento sotto rumore realistico a scala più grande.

Appendice: comandi Qiskit usati per la verifica

Codice

```
from qiskit.synthesis import TwoQubitWeylDecomposition, \
    TwoQubitBasisDecomposer
from qiskit.circuit.library import CXGate
from qiskit.quantum_info import Operator, process_fidelity
from qiskit import QuantumCircuit, transpile

# metodo 1: coordinate di Weyl di un'unitaria U (matrice numpy 4x4)
d = TwoQubitWeylDecomposition(Operator(U))
a, b, c = d.a, d.b, d.c    # c=0 -> 2 CNOT bastano; c!=0 -> servono 3

# metodo 2 (indipendente): sintesi forzata a N CNOT, verifica fedelta'
decomposer = TwoQubitBasisDecomposer(CXGate())
for n in [0, 1, 2, 3]:
    qc = decomposer(Operator(U), _num_basis_uses=n)
    print(n, process_fidelity(Operator(qc), Operator(U)))
print("minimo autonomo:", decomposer.num_basis_gates(Operator(U)))

# compressione automatica a CNOT minimi (per generare il circuito finale)
qc = QuantumCircuit(2)
qc.unitary(U, [0, 1])
compresso = transpile(qc, basis_gates=['cx', 'rz', 'sx', 'x'],
                      optimization_level=3)
```